

PARAKH — Stage 5 PRD

Risk Aggregation & Scoring Layer

1. Objective

Transform multiple anomaly signals into a **single, stable, interpretable risk score**:

\$\$

$R(r) \in [0,1]$

\$\$

This score must:

- Reflect **relative abnormality**
- Be **robust to scale and outliers**
- Avoid false precision (no unjustified parametric modeling)

2. Scope

Included

- Feature normalization (final)
- Rank-based aggregation
- Handling missing signals
- Risk score computation
- Risk distribution reporting

Excluded

- Confidence scoring (Stage 2)
- Routing decisions (Stage 6)
- Threshold calibration (only structure here)

3. Inputs

From Stage 4:

```
RiskFeatureResult.normalized_features
```

Each record has:

```
{
  "z_cost_norm": float | None,
  "hhi_norm": float | None,
  "burst_norm": float | None,
  "dup_norm": float | None
}
```

4. Output

Each record must produce:

```
{
  "risk_score": float
}
```

5. Mathematical Specification

5.1 Risk Score Definition

Let available normalized signals be:

\$\$
S(r) = \{s_1, s_2, \dots, s_k\}, \quad s_i \in [0,1]
\$\$

Then:

\$\$
R(r) = \frac{1}{|S(r)|} \sum_{s_i \in S(r)} s_i
\$\$

5.2 Key Properties

- Bounded: $(R \in [0,1])$
- Monotonic: increasing any signal increases risk
- Robust: unaffected by missing components

5.3 Missing Value Handling

If a signal is **None**:

- Exclude from aggregation

- Do NOT impute

If all signals missing:

```
R = None
```

5.4 Minimum Signal Requirement

If:

```
len(valid_signals) < k_min
```

Then:

- mark record as **insufficient data**

Default:

```
k_min = 2
```

6. Optional (Advanced, Not Required for SIH)

Weighted aggregation:

\$\$

$$R = \sum w_i s_i$$

\$\$

Constraints:

- $(\sum w_i = 1)$

Default: equal weights preferred

7. Implementation Architecture

7.1 Modules

- `aggregation.py`
 - `risk_score.py`
-

7.2 Core Class

```
class RiskAggregator:
    def __init__(self, k_min=2):
        self.k_min = k_min

    def compute(self, features):
        return RiskResult
```

7.3 Output Object

```
class RiskResult:
    scores: List[float]
```

8. Non-Functional Requirements

Stability

- No NaN or infinite values

Determinism

- Same input → same output

Performance

- < 1 second for 50k records
-

9. Validation & Testing

9.1 Sanity Checks

```
print(min(R), max(R))
```

Expected:

- Within [0,1]
-

9.2 Monotonicity Test

If:

- any signal increases

Then:

- (R) must not decrease
-

9.3 Missing Signal Test

- Drop one signal → score still computable
 - Drop all → None
-

9.4 Distribution Check

```
plot_histogram(R)
```

Expected:

- Not all values clustered
 - Some spread across range
-

10. Risk Bucketing (for interpretability)

Define:

```
LOW      = R < 0.3
MEDIUM  = 0.3 ≤ R < 0.7
HIGH     = R ≥ 0.7
```

Used only for reporting, not routing

11. Edge Cases (MANDATORY)

Handle:

- All signals missing
 - Only one signal available
 - All signals identical
 - Extreme skew (all near 0 or 1)
-

12. Acceptance Criteria

Stage complete if:

- ☐ Risk score computed for all valid records
 - ☐ Missing values handled correctly
 - ☐ Scores bounded in $[0,1]$
 - ☐ Output stable across runs
 - ☐ Distribution reasonable
-

13. Definition of Done

- Each record has a valid $R(r)$
 - Ready for Stage 6 (Decision & Routing)
-

14. Key Design Principle

Combine signals without pretending certainty.

This layer avoids:

- fake precision
- overfitting
- unjustified weighting

It produces a **clean, ordinal measure of abnormality**.
